CredoScore: Loan Default Risk Predictor

A one-page plain-English brief: what it does, how it was built, what was tested, and the questions a professor is likely to ask.

In one line: it is a website where you enter a loan applicant's details (age, income, loan amount, credit history, etc.) and it tells you how likely that person is to default, in plain numbers, with the reasons behind the number.

What the project actually is

- A web app called **CredoScore** (full name: *Loan Default Risk Predictor*).
- You fill in 12 simple details about a loan applicant (age, job, income, loan amount, how long the loan is for, credit history, etc.).
- It gives back three things: a **percentage chance of default**, a **plain recommendation** (approve / approve with conditions / review / decline), and a **list of reasons** for that number, ranked by how much each one mattered.
- There's also a "book" page that shows how the model performs across 1,000 past applicants, and a "methodology" page that explains how it was built and tested.

What data it learned from

- A public dataset from a German bank, collected in 1994: **1,000 real loan applications**, 700 that were repaid and 300 that defaulted.
- It's a well-known, standard dataset used worldwide to teach and test credit-risk models. It is not made up, not scraped, and it comes with an open licence.
- Out of the 20 details recorded per applicant, we kept the 12 most useful ones and deliberately **left out gender/marital status** so the model can't lean on it.

The 12 things it actually looks at

- Age
- How long they've had their current job
- Monthly income
- Loan amount requested
- How many months to repay it
- Number of other loans they already have
- Job type (skilled, unskilled, management, etc.)
- Housing (rent, own, or free housing)
- What the loan is for (car, education, business, etc.)
- Their past credit history (paid on time before? late before?)
- How much is in their checking account
- How much is in their savings account

Which "algorithm" we used, and why

- We didn't just pick one method and hope. We actually **tried four different prediction methods** and compared them fairly on the same 1,000 applicants, split into 5 equal groups so every method was tested the same way.
- The four: a simple scoring formula (logistic regression), a "committee of trees" that votes (random forest), two versions of a method called **gradient boosting**, and a well-known industry method called XGBoost.
- Gradient boosting won: it builds a few hundred very small, very simple decision trees, one after another, where each new tree focuses on fixing the mistakes of the ones before it. Added together, they make one strong prediction.
- We then fine-tuned its settings automatically (tried 40 different combinations) before locking in the final version.

How we actually built and run it

- The model was trained once, offline, using standard data-science tools (Python, pandas, scikit-learn).
- Instead of shipping that heavy software to the live website, we **exported the finished model as a small text file** (a list of simple yes/no questions, like a decision tree written out).
- The live website reads that file and answers each applicant with basic arithmetic, so no AI software needs to run live, which makes it fast, cheap to host, and simple to deploy.
- On top of the raw prediction, we added a calibration step so that when it says "30% chance of default," that number is trustworthy: roughly 30 out of 100 similar applicants really do default.
- It also breaks down *why* it gave that number, for example "loan amount pushed risk up 7 points, savings pushed it down 3 points", and those reasons always add up exactly to the final number, nothing is hidden or approximated.

Testing: what we ran, what broke, and how it got fixed

- We ran **56 automated tests** in total: 13 checking the model's maths is correct, 25 checking the website rejects bad or missing input properly, and 18 checking the actual screens work as expected. All 56 pass.
- **Something that broke and got fixed:** early on, the "why did it decide this" breakdown didn't add up to the right total. The cause was a rounding mistake in how we calculated each factor's contribution. A dedicated test caught it, we traced it to the starting point of the calculation being wrong, and fixed it so the numbers now add up exactly, every time.
- **Another fix:** the form let you pick "vacation" as a loan purpose, but the model was never trained on that category, so it would reject the request. We just removed that option from the form instead of guessing.

- **Another fix:** if the scoring service went down, two of the pages used to just spin forever with no error message. We changed them to load their data up front instead of over the network, so they now work even if that service is off, and the one page that still needs it shows a clear error instead of hanging.
- We also deliberately tried to break it: missing fields, negative numbers, huge loan amounts, nonsense categories. It always responds with a clear error message and never crashes.

Likely professor questions & plain-English answers

Q: Why gradient boosting and not a simpler method, or a fancier one like deep learning?
A: We tested a simple method (logistic regression) and it was noticeably worse. Deep learning needs a lot more data than our 1,000 applicants to be worth it, and it would've made the "why did you decide this" explanations much harder to trust. Gradient boosting was the sweet spot: strong accuracy, and every decision can be explained exactly, factor by factor.

Q: How good is the model, really?
A: On applicants it never saw during training, it scores 0.79 out of 1 on telling risky and safe applicants apart (1.0 would be a perfect psychic, 0.5 would be a coin flip). Its probabilities are also well-calibrated: when it says "X% chance," reality matches within about 2 percentage points.

Q: Is the dataset realistic for today?
A: It's from Germany in 1994, so the exact numbers (in old German currency) are dated. We say that openly on the site and in the report. The point of the project isn't "this exact model is ready for a real bank," it's demonstrating the full correct process: clean data, fair testing, calibrated probabilities, honest explanations, and proper deployment, which is the same process a real bank would use on their own current data.

Q: What happens if someone enters wrong or missing information?
A: The form checks everything twice, once in the browser and once again on the server, so you can't send incomplete or nonsense data even by tampering with the page. Every rejection comes back with a specific, readable reason, not a crash or a blank error.

Q: Is this fair to use, could it be biased?
A: We deliberately removed gender and marital status from what the model is allowed to see, as a precaution. We're upfront that we have not run a full, formal fairness audit beyond that, that's listed as a known limitation and as future work, not hidden.

Q: Where does it actually run, and what does it cost to host?
A: It's deployed on Vercel. The website is built with Next.js/React; the scoring itself runs as a tiny, dependency-free Python function, since it only needs to read that exported model file and do arithmetic, not run a full AI library. That keeps it fast and cheap to run.

Q: What would you improve if you kept going?
A: A proper fairness audit, training on more recent data, letting someone score a whole spreadsheet of applicants at once instead of one at a time, and adding monitoring so we'd notice if the model's accuracy drifted over time.